

# Evaluating the Auto-Vectorization Amenability of DaCe-Generated Code

Alexander Vali Bonsall  
ETH Zurich  
Switzerland  
abonsall@ethz.ch

Yakup Budanaz  
ETH Zurich  
Switzerland  
yakupkoray.budanaz@inf.ethz.ch

## Abstract

This semester project aims to improve the code emitted by DaCe by improving its vectorization rate among a sweep of different HPC computational patterns we evaluate amenability of the generated code on multiple HPC environments by evaluating on TSVC auto vectorization benchmark and selected loopnests from CloudSC. We inspect DaCe-generated code to assess how amenable it is to compiler auto-vectorization compared to human-written code, i.e. the base C++. From this analysis we provide insights on how to improve DaCe’s backend so that they (i) generate code that is more amenable to compiler optimizations such as cost models, and uncover important compiler flags that may be required. Concretely, we measure how many TSVC-2 kernels each production compiler, GCC and Clang, actually vectorizes on aarch64 and x86, and we determine why the remaining ones fail. We then repeat the analysis on CloudSC, a production weather-model benchmark, across its five extracted loop nests. Two questions remain open, does trying to force vectorization actually win performance? Or does the performance lie elsewhere? We show that with correct tuning of the code generation strategies we demonstrate a speedup of up to **1.7x**. Furthermore, we show that there are some fundamental problems with DaCe’s codegen and how it confuses compilers, especially GCC, at complex loops. This gets compounded once larger loop nests get involved.

## 1 Introduction

### 1.1 Motivation

Single-core performance on modern HPC hardware is dominated by how well a loop maps onto the SIMD units. The gap between scalar and fully vectorized code is close to an order of magnitude, and essentially all of that gap is left to the compiler’s auto-vectorizer to figure out. Auto-vectorization, as will be seen later within the paper, is rather brittle, it depends not on what a loop computes but on the exact shape in which the loop is presented to the compiler. Two programs with identical semantics can be interpreted by the compiler in varying degrees of accepted patterns. Purely because of how intermediates are named, where a value is stored, or whether a stride is a compile-time constant.

Thus the purpose of this Semester Project, that DaCe-emitted C++ is, as a starting point, roughly as vectorizable as the hand-written C++ or Fortran it replaces. If it is not, if the code generator systematically emits shapes that defeat the vectorizer, then an analysis of the code and uncovering the reasons as to why DaCe is emitting code that defeats compilers vectorization generation is necessary for DaCe’s future in the HPC landscape.

### 1.2 Related Work

TSVC was developed by Callahan et al. [1], consisting of 100 Fortran loops that have scalar semantics, and it has been used ever since as the benchmarking tool for measuring the vectorizer performance. Maleki et al. [2] converted it to C, expanded it, and discovered that production-level compilers were capable of vectorizing just a few loops; TSVC-2 [3] is a more refined C conversion of the benchmarking tool, and the suite keeps evolving

To that effect, Maleki et al. found GCC able to vectorize 59 of the 151 loops (39%) using its default cost model settings, compared to 90 by ICC and 68 by XLC [2]. In our experiments with the modern compilers, those numbers reflect the expected performance gain after ten years of research: with the default cost model settings, GCC 14 and LLVM 21 both vectorize 84/151 (55.6%) and 83/151 (55.0%) of the DaCe-emitted TSVC-2 kernels respectively, while relaxing that constraint causes GCC to achieve 99/151 (65.6%) of vectorization on aarch64 architecture, a ten percentage point increase due to profitability decisions alone at the expense of vectorizing previously unprofitable loops. To prove this: Pohl et al. find that those profitability estimates correlated with the performance gain only mildly ( $\rho = 0.58$  for LLVM and 0.33 for GCC AVX2) [4].

However, Siso et al. [5] argue that TSVC gives compilers almost perfect data (e.g., `restrict`, alignment constraints, and static bounds), which is why it represents an ideal case and is progressively withdrawn. On the other hand, Pohl et al. [4, 6] prove that the cost models behind GCC and LLVM vectorizers underestimate speedup and poorly correlate with the gain achieved.

### 1.3 Contributions

This Semester Thesis will attempt to contribute and report the following:

- We compile all 151 TSVC-2 kernels as hand-written C++ and as DaCe-emitted C++, under GCC and Clang/LLVM, and record which loops each combination vectorizes. DaCe-emitted code vectorizes fewer loops than the C++ ports under both compilers.
- For a set of kernels that vectorize from C++ but not from DaCe (s313, vdotr, s453, s314), we read the emitted C++ and the resulting assembly to identify *why* the vectorizer refuses, and we confirm each diagnosis by editing the generated code minimally until it vectorizes.
- We measure execution time for the original, DaCe, and hand-edited DaCe variants, and show that vectorizing and being fast are not the same property: several kernels vectorize and get slower.
- We repeat the analysis on CloudSC, a production weather-model benchmark, as a way of having a real-life use case, across its five extracted loop nests, two compilers, and two cost models, and

we identify a Fortran-to-Python porting defect that had been depressing the Python-frontend results.

- We boil down the findings into concrete, actionable changes to DaCe.
- The Semester Project is making establishing that we work on the C++ that DaCe emits, *without* running the DaCe auto-vectorization pipeline. The question here is what the code generator hands to the compiler by default, not what DaCe can achieve when its own optimization passes are enabled.

## 1.4 What is TSVC-2

TSVC kernels are a collection of different simple loops of code that represent a suite of different HPC environments. The suite contains loops with scalar semantics organized in four main categories: dependence analysis, vectorization, idiom recognition, and language completeness.

## 1.5 What is CloudSC

CloudSC is a standalone mini-app extracted from the cloud microphysics parameterization scheme of the European Centre for Medium-Range Weather Forecasts Integrated Forecasting System [7].

It is widely used to test and optimize weather-prediction code across CPUs and GPUs, and it serves here as the real-world counterpart to TSVC-2's synthetic loops. CloudSC spans over 2 000 lines of Fortran, which is too large to analyse as a single unit. It was therefore broken into five representative loop nests that between them exercise the computational patterns appearing throughout the full application, these are: Autoconversion snow, Ice supersaturation adjustment, LU solver, Rain evaporation (Abel-Boutle), and Saturation calculation.

The first loops, "Saturation Calculation" diagnoses the saturation of humidity, the results are then funneled into "Ice Supersaturation adjustment" which checks for excess vapor which would be turned into rain or ice and "Autoconversion snow" to see if the ice in the clouds will turn into snow. Eventually falling as rain/snow, which then leads into "Rain evaporation abel boutle" to see how the liquid which fell would evaporate.

Each loop has its loop characteristics, "Autoconversion Snow" is a flat 1D loop, two nested scalar IFs, no cross-column dependence, which happens to be incredibly parallel. "Ice Supersaturation Adjustment" is also a flat 1D loop, but has several branches, no cross-column coupling. "Lu Solver" is the most complex of the bunch, with a loop-carry dependency. "Rain Evaporation Abel Boutle" is a mix of the first two, with a flat 1D loop, single gating branch, which again is incredibly parallel. The last "Saturation Calculation", the only kernel with a genuine 2D loop nest with no dependencies. These too are iterated over the different combinations as in tsvc.

## 2 Methodology

### 2.1 Compilers, Cost-models and Architectures

The main goal of the thesis is to get a good understanding of where compilers on equivalent human written C++ or Fortran was vectorized by compilers, while DaCe-ingested Fortran or DaCe-ingested Python failed. From there it is then understood what DaCe is still lacking in and room for improvement. Thus, the need of a large sweep of different environments and situations.

**2.1.1 Architectures.** Again, a similar argument to GCC vs Clang, is the Aarch64 vs x86 architectures. Both once again being large dominating choices for HPC.

**2.1.2 Cost-Models.** An interesting point of view for DaCe was to see what difference force vectorization would do to performance and trying to find a sweetspot of which cost-models would be most profitable. DaCe's goal is to improve computation towards a more data-centric paradigm shifts and thus larger code can be computed in a more efficient manner. Cost-models and their influence on the vectorization of the compiled code and DaCe's added influence needs to be studied and understood where the most effective cost-model is. That being said, the 4 cost models that will be in this paper are named as such:

- Disabled: The compiler will not allow for any vectorization.
- Default: The most conservative cost-model, only allowing vectorization if there is a clear and positive benefit.
- Cheap: Compiler allows the vectorization even if it believes it will have the same performance as non-vectorized code.
- Unlimited: Compiler forces vectorization, even if it harms performance.

All the different cost-models have a similar set of Flags: `-O3, -std=c++17, -fPIC, -fno-math-errno, -ffinite-math-only, -fno-signaling-nans, -fstrict-aliasing, -faligned-new`

There are many more cost-models supplied by the compilers, but these will be the ones tested. For Clang/LLVM the `-O3` flag will already be baked in default cost-model, no need to change anything there. The disabled model required that the `-O3` flag be replaced with the `-O0` flag, since LLVM will prefer `-O3` over a `-fno-vectorize`. Default had no extra added flags and stayed to what the `-O3` flags allowed. Unlimited added an extra `"-fvector-cost-model=none"` such that LLVM was forced to vectorize. The cheap model had a `"-mprefer-vector-width="`, which was depending on architecture would have different values; `aarch64(arm_grace)=128` and `x86(amd_epyc)=256`. Although, cheap was difficult to work with and would later be dropped as preliminary results showed little difference. GCC was much simpler, as it already had a baked in `"-fvect-cost-model"` for `cheap=cheap`, `default=dynamic`, `unlimited=unlimited`. Unlike LLVM, `"-fno-tree-vectorize"`, `"-fno-tree-slp-vectorize"` works when `-O3` is established.

### 2.2 Cluster

We test on two vClusters of the CSCS Alps system [8]. Daint [9] provides the aarch64 target: each node holds four NVIDIA GH200 modules, whose Grace CPUs contribute 72 Arm Neoverse V2 cores

each (288 per node) with four 128-bit SIMD units per core [10]. Eiger [11] provides the x86-64 target: dual-socket AMD EPYC 7742 (Zen 2), 64 cores per socket, whose widest vector extension is AVX2 at 256 bits.

## 2.3 Work Flow

To get the data for the vectorization reports for TSVC-2 a script was designed that iterated over all the TSVC-2 kernels over all the combinations of cost-model, architecture and compilers. At each run, flags for full compiler reports were saved to be used as raw data for the results, as well on the side the assembly files were dumped for deeper understanding.

Similarly for CloudSC, there are 3 different versions, the original fortran, DaCe based on python and python base. In all these cases python is also Numpy. Original fortran will play as the baseline for the CloudSC analysis. Fortran side: `MATMUL`, `DOT_PRODUCT`, `SUM`, `TRANSPPOSE` are language intrinsics, the compiler frontend recognizes the call site itself (not the loop nest it would otherwise expand to) and can lower it directly to an optimized routine. gfortran's `-fexternal-blas` (with `-fblas-matmul-limit=N`) literally replaces `MATMUL` calls above a size threshold with a call to `dgemm/dgemv` from a linked BLAS, the vectorizer never even sees a loop to analyze, because there isn't one in the generated IR.

Once the builds were completed and the vector reports were saved, a second run would be performed, this time for a run-time measurement. To see if the vectorization won and sped-up the loop. If knowledge of the assembly was needed for the CloudSC loops, the `.so` files could be objdumped.

To do these runs, Daint and Eiger Clusters were used, setting up a uenv and venv with the necessary GCC and Clang/LLVM compilers (The version of LLVM is 21.1.4 and GCC 14.3.0), as well as adding the fortran compilers (Flang and Gfortran) through Spack. Connecting the DaCe repository and necessary dependencies to said venv were the final steps in order to ensure the environment was correctly established.

From there many different source files were generated for each of the combinations. These would be called using `SOURCE` to change the compile flags for the specific run. During the first calls of the scripts all the necessary SDFGs were generated on the cluster to ensure that they would be correctly established for that architecture.

## 3 TSVC 2

### 3.1 Vectorization of TSVC

At the beginning of the study there were some preliminary results done to see what combinations were worth while to look at. It seemed the different Cost-models had very little difference, due to time constraints it was decided to drop most of them except unlimited, to try and maximize the vectorization, but note that default and cheap had almost the same amount of loops vectorized, see figure: 1. Similar decisions were made with the different CPU types, as the available x86 CPU on the Eiger cluster was significantly slower (and older) than the aarch64 CPU available on the Daint cluster, which also resulted in slightly less vectorization as well. Resulting in only unlimited cost-model on the Daint cluster (aarch64/arm\_grace) being the only avenue of meaningful approach.

Vectorization rate — TSVC-2  
DaCe-generated code, double precision

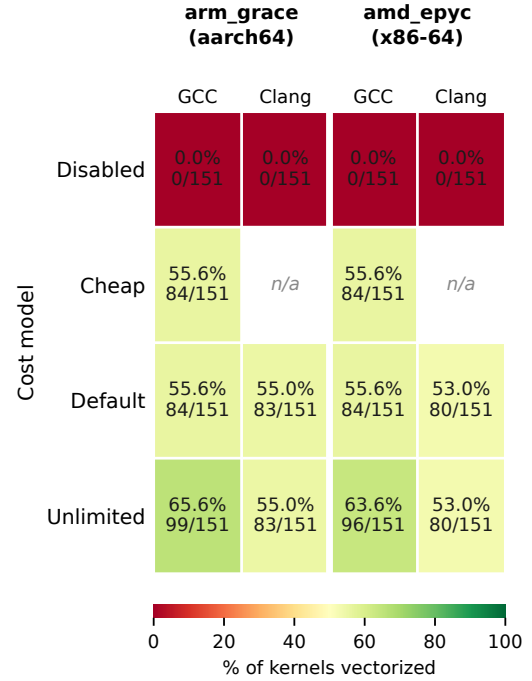


Figure 1: Heatmap of the different compiler, cost-model, CPU combinations. To show that the different cost-models had very little influence, these are preliminary results

After running all 151 TSVC-2 kernels and checking to see which kernels vectorized and which did not within an environment of cost-model unlimited within GCC & Clang, using C++ as a baseline.

Clearly DaCe under-performs vs the C++ ports of TSVC2 kernels when it comes to the amount of loops it can successfully vectorize, on this figure: we can see that GCC+DaCe specifically fails on 8 kernels where every other combination succeeded, and 11 cases where only C++ succeeds.

|             | Only DaCe | Only C++ | DaCe fails | C++ fails | Combined | Both fail |
|-------------|-----------|----------|------------|-----------|----------|-----------|
| GCC         | 2         | 4        | 8          | 1         | 8        | -         |
| Clang       | 0         | 2        | 1          | 0         | 7        | -         |
| GCC + Clang | 2         | 5        | -          | -         | 82       | 29        |

Figure 2: Amount of kernels vectorized with certain combinations, only 25 kernels had an area of improvement, essentially everything left of the combined column. Only GCC+DaCe, means only that specific combo worked, i.e. the 3 other combinations failed on those 2 kernels and GCC + DaCe fails, means that in those 8 kernels only DaCe failed for GCC, the other 3 combinations vectorized

### 3.2 Reasons for not vectorizing

Looking deeper into the tsvc-2 kernels. A few selected kernels were picked that managed to vectorize in the C++ kernels but not for DaCe. Here is an example of the patterns in which combinations failed or passed figure: 2.

3.2.1 *s313 (Dot Product)*. This TSVC-2 kernel is a simple dot product.

```

1 void s313_d_single(const double * a, const double * b,
2   ↳ double * dot, int iterations, int len_1d)
3 {
4     dot[0] = 0.0;
5     for (int i = 0; i < len_1d; ++i) {
6         dot[0] += a[i] * b[i];
7     }

```

Figure 3: s313 (Dot Product)

This one is rather simple, GCC does not recognize that the `dot[0]=__out;` (on line 14 at figure: 4) could be added outside of the loop. GCC should change `dot[0]=__out` to a static value instead of calling every loop from memory. Another option would be if DaCe could take that outside the loop the complex access pattern concern from GCC should be resolved.

```

1 void __program_s313_d_single_s313_d_single_internal(...)
2 {
3     ...
4     for (i = 0; (i < LEN_1D); i = (i + 1)) {
5         ...
6         {
7             //multiplication of a[i] * b[i] ...
8             //addition of multiplication
9         }
10        {
11            double __inp = add_of_mul;
12            double __out;
13            __out = __inp;
14            dot[0] = __out;
15        }
16    }
17 }

```

Figure 4: s313 (Dot Product), DaCe transformed, showing how DaCe takes the individual calculations and separates them into different tasklets

Testing different variants of this DaCe generated base C++ code, the most important factor is changing the `dot[0]` to a scalar temporary value that does not have to be called every time from memory and thus have no complex access pattern. Moving the final summed value save to outside the loop, keeps the semantic correctness of the code. With these few adjustments it becomes vectorized. In other words, DaCe needs to recognize possibilities for turning consistent variables into a single temporary variable and saving it outside of the loop.

**3.2.2 vdotr (Dot Product, with Loads/Stores Hoisted).** Vdotr is a TSVC-2 vector dot product kernel but with Loads/Stores Hoisted (named after the LLVM naming convention [12], i.e. taking an array access/write from within the loop out of it, such that no write instructions are needed at ever loop iteration), almost exactly the same as s313.

With this kernel DaCe optimization dragged the summed value that should be saved to the vector into the loop. This drastically increases the pipeline due to constant save instructions. Not only is this inefficient, it is completely unnecessary. DaCe tried to vectorize everything within the kernel to the point where it became detrimental. Taking the `dot_out[0]=__out;` outside the for-loop

```

1 void __program_s313_d_single_s313_d_single_internal(...)
2 {
3     int64_t i;
4     dot[0] = 0.0;
5     double dot_slice_plus_a_slice_b_slice = dot[0];
6     for (i = 0; (i < LEN_1D); i = (i + 1)) {
7         //multiplication of a[i] * b[i] ...
8         //addition to dot_slice_plus_a_slice_b_slice ...
9     }
10    {
11        double __inp = dot_slice_plus_a_slice_b_slice;
12        double __out;
13        __out = __inp;
14        dot[0] = __out;
15    }
16 }

```

Figure 5: s313 (Dot Product), DaCe improved, taking the hot array access outside of the loop, such that only one store instruction is needed for the calculation

```

1 void vdotr_d_single(doubles and floats) {
2     double dot = 0.0;
3     for (int i = 0; i < len_1d; ++i) {
4         dot += a[i] * b[i];
5     }
6     *dot_out = dot;
7 }

```

Figure 6: vdotr (Dot Product, with Loads/Stores Hoisted)

immediately allows GCC to vectorize the loop. This is a similar situation to the figure: 4 and similar improvement of figure: 5.

### 3.3 S313 (Dot Product) & Vdotr (Dot Product, with Loads/Stores Hoisted) execution run results

An important detail to note is that DaCe's built-in timing counter reports at microsecond granularity, nanosecond results are discarded and thus have a less accurate result. For the TSVC-2 kernels, whose runtimes are in the thousands of microseconds, this is issue can be ignored.

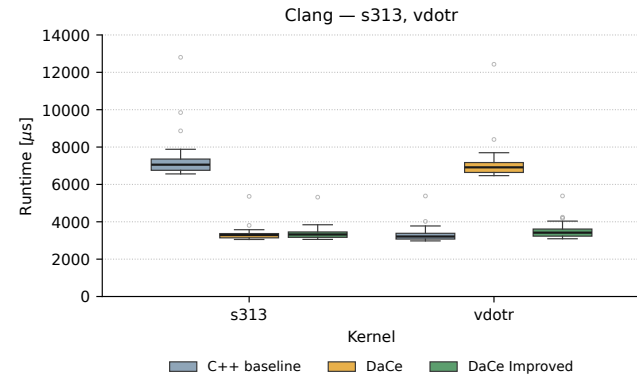
The execution run results for the Clang variation was close to what was expected, figure: 7a. Solid performance 2.15x(median comparison) improvement for s313 with the improved edited version of DaCe having no effect on Clang, likely due to clang already recognizing the hot loop access and optimizing for it.

A nice surprise was the improvement on vdotr (Dot Product, with Loads/Stores Hoisted) where DaCe came on par to the C++ baseline. Seemingly DaCe vectorized, as reported in both the compiler report and from assembly check, and yet had worse performance. After checking the assembly even closer for this specific result, its likely due to Clang+CPP used `fmla` (floating multiply addition) instructions vs Clang+DaCe which used a sequence of `fmul` & `fadd`, which is essentially the previous just separated.

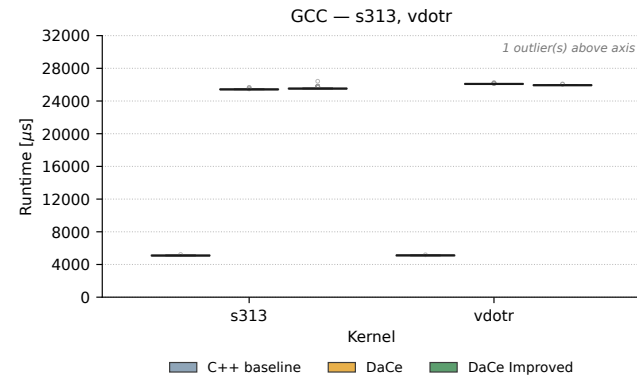
This will essentially double the FP cycles to the whole pipeline and likely also the culprit for the close to doubling run execution. Digging even deeper in, clang only generates `fmla` when within the C++ file the code is written as `dot+=a[i]*b[i]` and because of DaCe's dataflow decomposition the code is separated into its

individual parts `dot=dot+tmp` which clang does not fuse, unless a specific flag is used `-ffp-contract=fast` [13].

On the other-hand, GCC is rather a surprise, even if the DaCe improved code was used the improvement of performance was negligible at best, see figure: 7b. It seems that GCC and DaCe have some sort of underlying compatibility issue. Likely from DaCe's data flow decomposition. Due to GCC being more strict on its pattern recognition the DaCe layout is being restricted and compiled in serial.



(a) Boxplot of the Clang/LLVM s313 & vdotr execution time results, showing the improvement increase in performance from vdotr



(b) Boxplot of the GCC s313 & vdotr execution time results, showing that even though DaCe improved had managed to successfully vectorize the loop, there was no performance increase

Figure 7: Comparison of s313 and vdotr execution times across Clang/LLVM and GCC

**DaCe improvement 1:** DaCe needs to stage through scalar variables as it is optimized better by compilers, instead of the current repeated writes to global arrays within loops. As well as, rethink the data-flow decomposition, or specify required important flags.

### 3.4 Vectorizing s314(Max Redcution) & s453 (Induction Variable Vectorization)

3.4.1 s453 (Induction Variable Vectorization). This is the TSVC-2 induction variable vectorization test.

GCC does not vectorize this loop because of the additional copies DaCe emits around the induction variable: folding/removing them

```
1 void s453_d_single(doubles ...) {
2     double s = 0.0;
3     s = 0.0;
4     for (int i = 0; i < len_1d; ++i) {
5         s += 2.0;
6         a[i] = s * b[i];
7     }
```

Figure 8: s453 (Induction Variable Vectorization)

requires reordering the sequence of floating-point additions, and GCC's vectorizer treats that reordering as illegal unless floating-point reassociation is explicitly permitted. This is clear when adding `-funsafe-math-optimizations`, which also enables the specific sub-flag `-fassociative-math`, which allows GCC to reassociate chains of floating-point operations at the risk of small FP errors. With `-fassociative-math` enabled, the copies can be folded and the GCC DaCe kernel vectorizes. This is compounded by the Clang DaCe analysis, which vectorizes the loop without enabling any unsafe-math flag at all: instead of needing to fold DaCe's copies away, Clang unrolls the loop and generates vectors of the `s+=2.0` induction directly, pipelining the unrolled, vectorized adds. This lets Clang vectorize the loop without reordering or reassociating any FP operations, and so without risking FP error.

Checking the GCC+C++ version, which happens to vectorize, the pattern in the assembly code between the GCC+C++(no -funsafe) and GCC+DaCe (with -funsafe) have a semantic similarity. Meaning that the -funsafe flag that would allow for reordering of FP operations, did in actuality the compiler did not do any FP reordering, instead it vectorized in the same way as the C++ version did. In this situation, the flag is actually buying GCC's willingness to *attempt* the transformation at all. Making GCC tree more inclined to behave closer to clangs vectorization analysis. This could be due to DaCe's dataflow decomposition again and it negatively influencing GCC's pattern recognition.

3.4.2 s314 (Max Reduction Loop). A TSVC-2 max reduction loop.

```
1 void s314_d_single(doubles ...) {
2     double x;
3     x = a[0];
4     for (int i = 0; i < len_1d; ++i) {
5         if (a[i] > x) {
6             x = a[i];
7         }
8     }
9     result[0] = x;
10 }
```

Figure 9: s314 (Max Reduction)

In this situation the DaCe compiled version had a similar story of the s453. This kernel needed slight adjustment due to complex access patterns developed from the `a[i]` array access, two ways to solve it was either replacing the DaCe copy function `'dace::copy(...)'` with a simple `x=a[0]`. This is due to DaCe emitting a copy function for all copies, even if they are for single element access, this then get lowered to `'memcpy'` IR-native lowering which optimizes differently. By having an assigned operator for a single-element access enables the compilers for more optimizations. The other option is by adding buffering variables into the for-loop to have an

internal loop variable. Either of these options lead to solving the complex access pattern. The next issue was the same story with the previous kernel. The need for the `-funsafe-math-optimizations` flag solved GCC's problem of a stmt unrecognized variable pattern.

### 3.5 s314(Max Redcution) & s453 (Induction Variable Vectorization) execution run results

Starting with the Clang side, s314 (Max Reduction) has a win in performance almost to the level of C++ but with large varying values. Looking at the tracing figure: 11, we can see that the execution time start stabilizing later on in the iterations, around run 50. Which is likely why the box graphs are so tall for the DaCe-improved.

s453 (Induction Variable Vectorization) on the other hand decreased in performance with the DaCe improvement by almost double. Indicating a situation where forcing vectorization does not always lead to improved performance.

The biggest win in the semester project has been with kernel s314+GCC and the **1.7x** increase in performance over C++ and DaCe baseline as seen on figure: 10b. The interesting detail is that the GCC+C++(which vectorized) and GCC+DaCe base (which did not vectorize) are about on par with each other in performance. Looking deeper into the assembly output there seems to be no semantic difference between the two. Perhaps due to a warm cache or the randomized array generated for the max reduction loop being favorable branches for the DaCe improved. There should be further studies done on heavily branching loops within DaCe.

**DaCe improvement 2:** Due to potentially DaCe's data-flow decomposition certain flags like `-funsafe-math-optimizations` which allows for `-fassociative-math` flag allow FP operations to be reordered and thus buying GCC's willingness to vectorize. DaCe & GCC may need to always allow FP flags, or a redesign emitting less connectors is necessary.

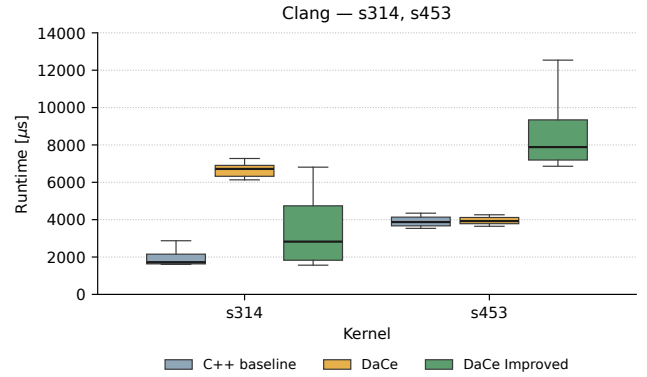
## 4 CloudSC

### 4.1 CloudSC discoveries

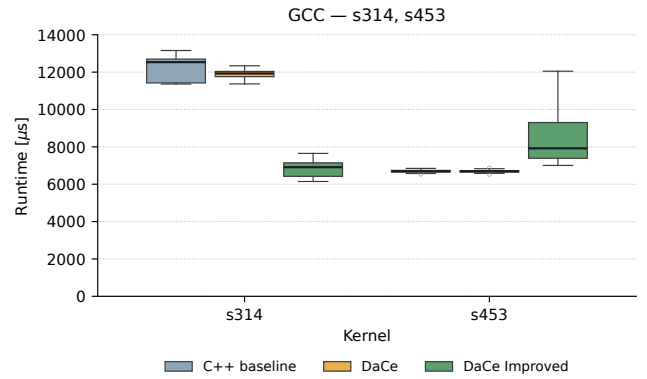
Initial measurements had shown that the kernel Lu\_solver Python frontend was **2x slower** than the original fortran and the DaCe fortran frontend. This was due to a porting bug from fortran to python, with the row-major to column-major mix-up, which lead to a inefficient access pattern. This also coincidentally led to DaCe generating code that made variables within the loop become run-time. The NCLV\*NCLV on line 7 or 11 of the figure: 12 shows where the run-time was within a triple nested loop. This is exactly what defeated both LLVM/Clang & GCC auto-vectorizer due to a run-time stride calculation.

To solve this issue a simple reverse swapping of the indexes within all the python DaCe codes. This regenerated the code with a compile-time constant stride, allowing the compiler LLVM to vectorize the loop, as can be seen in figure: 13.

This drastically improved performance, by a factor of **8.4x**. There was also a similar performance improvement for python in the other problematic kernel, ice\_supersaturation\_adjustment and saturation\_calculation, bringing the python kernel performance up to

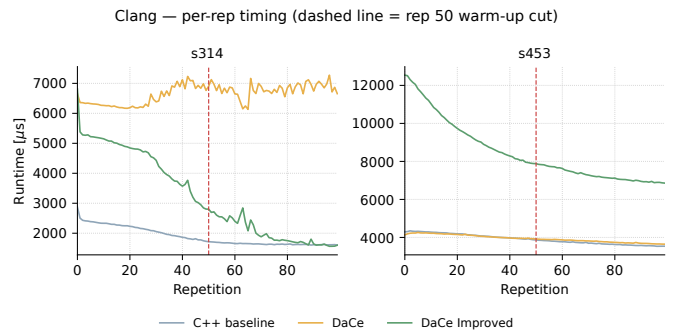


(a) Boxplot of the Clang/LLVM s314 & s453 execution time results, showing the improvement increase in performance from s314 and the decrease in performance from s453



(b) Boxplot of the GCC s314 & s453 execution time results, showing the improvement increase in performance from s314 and increase instability of s453

**Figure 10: Comparison of s314(Max Reduction) and s453(Induction Variable Vectorization) execution times across Clang/LLVM and GCC, with the before and after of improvements to DaCe**



**Figure 11: Run time execution traces of s314 & s453, here with the `-funsafe` flag applied the kernels require some 50 loops warmup time to stabilize**

par with the other Fortran basic and DaCe Fortran. A comparison can be seen between figure: 14 and 16.

```

1 void __program_lu_solver_python_frontend_internal(...)
2 {
3     for (jn = 0; (jn < (NCLV - 1)); jn = (jn + 1)) {
4         for (jm = (jn + 1); (jm < NCLV); jm = (jm + 1)) {
5             for (jl = 0; (jl < KLON); jl = (jl + 1)) {
6                 dace::CopyND(...)y(
7                     zqlhs + (((NCLV * NCLV) * jl) +
8                         (NCLV * jm)) + jn), &zqlhs_index, 1);
9
10                dace::CopyND(...)y(
11                    zqlhs + (((NCLV * NCLV) * jl) +
12                        (NCLV * jn)) + jn), &zqlhs_index_0, 1);
13            }}}}
14 }

```

Figure 12: Lu\_solver python DaCe generated code with the run-time variable within the hottest/nested loop

```

1 {
2     double __inp = zqlhs_slice_div_zqlhs_slice;
3     double __out;
4     __out = __inp;
5     zqlhs[(((KLON * NCLV) * jn) + (KLON * jm)) + jl] =
6     ↪ __out;
7 }

```

Figure 13: Lu\_solver python DaCe generated code with the solved data accesses, no longer requiring the DaCe::Copy function due to fixed array access stride

| Before Fortran → Python fix |                 | Autoconversion<br>_snow | Ice_super-<br>saturation<br>_adjustment | Lu_solver | Rain<br>_evaporation | Saturation<br>_calculation |
|-----------------------------|-----------------|-------------------------|-----------------------------------------|-----------|----------------------|----------------------------|
| DaCe +<br>Fortran           | clang_Default   | 0                       | 0                                       | 6         | 0                    | 1                          |
|                             | clang_Unlimited | 1                       | 0                                       | 6         | 1                    | 1                          |
|                             | gcc_Default     | 0                       | 0                                       | 6         | 0                    | 0                          |
|                             | gcc_Unlimited   | 0                       | 0                                       | 6         | 0                    | 0                          |
| DaCe +<br>Python            | clang_Default   | 0                       | 0                                       | 0         | 0                    | 0                          |
|                             | clang_Unlimited | 0                       | 0                                       | 2         | 0                    | 1                          |
|                             | gcc_Default     | 0                       | 0                                       | 0         | 0                    | 1                          |
|                             | gcc_Unlimited   | 0                       | 0                                       | 6         | 0                    | 0                          |
| Fortran                     | clang_Default   | 0                       | 0                                       | 6         | 0                    | 0                          |
|                             | clang_Unlimited | 0                       | 0                                       | 6         | 0                    | 0                          |
|                             | gcc_Default     | 0                       | 0                                       | 6         | 0                    | 0                          |
|                             | gcc_Unlimited   | 0                       | 0                                       | 6         | 0                    | 0                          |
| Total Loops                 |                 | 1                       | 1                                       | 6         | 1                    | 1                          |

Figure 14: Heatmap of all the vectorization for CloudSC, before the fortran→Python fix

## 4.2 Vectorization of CloudSC

Similar to the TSVC-2 benchmarking, the difference in cost-model and different CPU types had very little difference in vectorization, and in some cases for CPU type the x86/amd\_epyc was significantly slower, as can be seen in figure: 15. Grace kernels are, on average, roughly 1.64× faster than equivalent kernels running on a single-socket amd\_epyc CPU. This performance improvement is consistently observed and aligns with the practical differences in memory bandwidth per node. Theoretically, a single-socket EPYC CPU provides up to 204.8 GB/s of peak bandwidth<sup>1</sup>, whereas the

Nvidia Grace CPU utilizes LPDDR5X memory to achieve 512 GB/s<sup>2</sup>. While this represents a theoretical peak ratio of 2.5×, the observed 1.64× speedup (derived from kernel metrics of 450μs on EPYC versus 275μs on Grace as seen on figure: 15) closely reflects the sustainable bandwidth advantage of the Grace architecture in real-world workloads. For more details there is a heatmap in the appendix figure: 20. Thus the reason for dropping the amd\_epyc results, as the highest performance are what we are trying to improve upon. To continue the Default and Unlimited were chosen to be compared against, due to time constraints and that there were close similarities with TSVC-2.

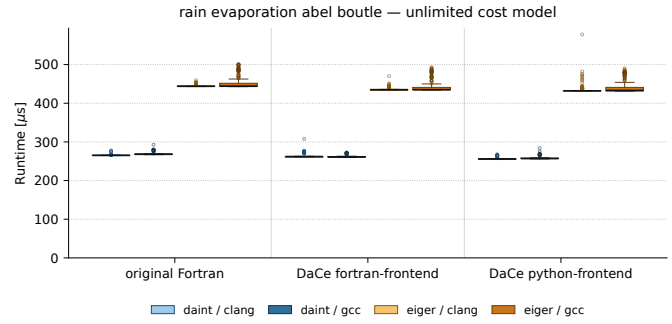


Figure 15: Example of Rain evaporation CloudSC loop that shows Eiger/aarch64/amd\_epyc being significantly slower due to it being older

Not all kernels could be vectorized, Ice\_supersaturation.. could not be due to its very branching loops. Autoconversion\_snow and Rain\_evaporation.. are both also branching loops, although not as drastic as Ice, thus when forcing vectorization through with cost-model unlimited, there is some vectorization as seen in figure: 16. The improvements to DaCe + Python in Lu\_solver are due to what was discovered and fixed from 4.1. The trick used in 3.5 does not sadly work in any of the CloudSC cases, as GCC in these cases is not having difficulty reducing the values and thus giving more reason to vectorize, due to the flag, is having no effect on GCC.

## 4.3 Execution Time of vectorized CloudSC

Again as another note, is that DaCe's built-in timing counter reports at microsecond granularity. For the shortest CloudSC loops it is a problem: differences of one or two microseconds sit at the resolution limit of the instrument and are reported here as such rather than as findings.

Although vectorizing as much as possible has been possible with some kernels, it is not always beneficial. Vectorization may sometimes take too long to load large vectors and thus hit memory too hard and increase latencies, which would be offset by the quick compute time. But when the loops are too low it may be easier to simply serialise it. A good example is figure: 17 (when compared with figure: 16, as one can see only the unlimited clang are vectorized for Autoconversion snow), with the unlimited being **dramatically** slower than its default counterpart. Another such case is in Lu\_solver on figure: 19 within the appendix.

<sup>1</sup>AMD EPYC 7742 Specifications: <https://www.amd.com/en/products/cpu/amd-epyc-7742>

<sup>2</sup>NVIDIA Grace CPU Whitepaper: <https://resources.nvidia.com/en-us-grace-cpu/nvidia-grace-cpu-whitepaper>

|                |                 | After Fortran<br>→<br>Python fix |                 | Autoconversion<br>snow |               | Ice_super-<br>saturation<br>adjustment |                 | Lu_solver   |               | Rain_evaporation |                 | Saturation<br>calculation |               |
|----------------|-----------------|----------------------------------|-----------------|------------------------|---------------|----------------------------------------|-----------------|-------------|---------------|------------------|-----------------|---------------------------|---------------|
|                |                 | Clang_Default                    | Clang_Unlimited | GCC_Default            | GCC_Unlimited | Clang_Default                          | Clang_Unlimited | GCC_Default | GCC_Unlimited | Clang_Default    | Clang_Unlimited | GCC_Default               | GCC_Unlimited |
| DaCe + Fortran | Clang_Default   | 0                                | 0               | 6                      | 0             | 1                                      |                 |             |               |                  |                 |                           |               |
|                | Clang_Unlimited | 1                                | 0               | 6                      | 1             | 1                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Default     | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Unlimited   | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
| DaCe + Python  | Clang_Default   | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
|                | Clang_Unlimited | 1                                | 0               | 6                      | 1             | 1                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Default     | 0                                | 0               | 6                      | 0             | 1                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Unlimited   | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
| Fortran        | Clang_Default   | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
|                | Clang_Unlimited | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Default     | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
|                | GCC_Unlimited   | 0                                | 0               | 6                      | 0             | 0                                      |                 |             |               |                  |                 |                           |               |
| Total Loops    |                 | 1                                | 1               | 6                      | 1             | 1                                      |                 |             |               |                  |                 |                           |               |

Figure 16: Heatmap of all the vectorization for CloudSC, after the fortran-→Python fix, Lu\_solver fully vectorizes across all iterations and clang unlimited for DaCe+Python vectorize too

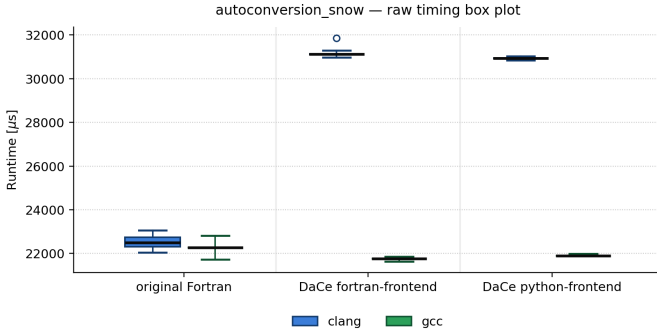


Figure 17: Boxplot of Autoconversion\_snow with unlimited cost-model

Rain\_evaporation is a kernel which uses a lot of max and min functions. Figure: 18 also shows a slight slowdown in comparison to the default Fortran, specifically for GCC. Which is also what rule: 3.5 is an improvement for. If this were to be applied here, there is a likely chance for a speed up.

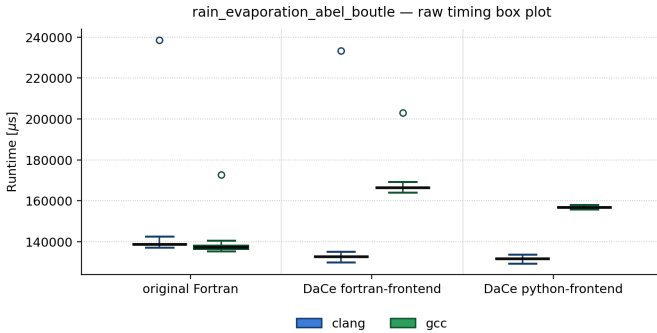


Figure 18: Boxplot of Rain\_evaporation with unlimited cost-model, showing how GCC on DaCe fortran-frontend is worse than clang, due to potentially what rule:3.5 mentioned

## 5 Conclusion

### 5.1 Summary of findings

DaCe-emitted C++ is measurably less amenable to auto-vectorization than the hand-written C++ it replaces. Across the 151 TSVC-2 kernels, the C++ ports vectorize **106 loops** under Clang and **108** under GCC, against **100 and 96** for the DaCe-emitted versions.

Looking into the specific kernels where there was room for improvement in vectorization. In the s313(Dot Product) and vdotr(Dot Product, with Loads/Stores Hoisted), DaCe writes the accumulator to memory on every loop, **simply moving the hot access variable enables vectorization**. In s453 and s314, the extra copies DaCe emits around induction variables and single-element accesses prevent GCC from even attempting the transformation, and a single flag (`-funSAFE-math-optimizations`, via `-fassociative-math`) restores it.

s313 saw no run time execution improvement from vectorizing, across GCC and Clang. vdotr had equally no influence from vectorizing its GCC loops, but the same improvements did have an influence on the already vectorizing clang loops bringing it to the same performance as C++.

s314 saw great improvements from allowing the `-funSAFE` flag, while keeping result accuracy and keeping a semantic equality to the pre-improved DaCe in FP operations. s453 in this specific case degraded in performance, without a clear reason why.

CloudSC had a large row/column-major Fortran-→Python bug that was solved, and then further look into the results showed that forcing vectorization lead to little improvement, and in some cases lead to reduced performance. DaCe must keep an eye on what it generates as the bloat it adds makes compilers less likely to vectorize, due to them not recognizing patterns.

### 5.2 Vectorising is not the same as being fast

The Clang+DaCe s313 variant is 2.15× faster than the C++ baseline while the hand-edited "improved" variant changes nothing measurable, but at the same time did not negatively impact performance. Under Clang, the same edit that makes s453 vectorize makes it roughly 2× slower. And in the clearest case, GCC+C++ (which vectorizes) and GCC+DaCe (which does not) run s314 at essentially the same speed, with no semantic difference visible in the emitted assembly.

Any tool that uses "did it vectorise?" as its objective, including a variant search built on DaCe, will therefore be optimizing a proxy that disagrees with wall-clock time often enough to matter. Thus, a need for a more indepth look under the hood of the code, checking assembly and memory access patterns.

### 5.3 Recommendations for DaCe

The improvement rules stated in the body reduce to three code-generator changes:

- (1) **Stage reductions through scalar temporaries.** Accumulators should live in a local variable for the duration of the loop and be written back once the loop ends. The present pattern of repeated writes to a global array inside the loop is the single most frequent cause of the failures.

- (2) **GCC and DaCe require more FP flags.** DaCe’s Data-flow decomposition is strangling GCC’s ability to recognize patterns, allowing certain FP flags, like `-funsafe-math-optimizations` placates GCC to vectorize. The second solution is to redesign DaCe to emit less connectors.

## 5.4 Possibilities for future work

Warm-up is not fully accounted for. Several variants drift monotonically downward across the measured repetitions, and for the Clang s314 and s453 DaCe-improved variants that drift is large enough that the reported speedup depends on which repetitions are included. The s453 case had not reached a steady state by the end of the run at all. There is potential for more accurate data and longer run times with larger arrays in general that could possibly help with uncovering further details in vectorization within DaCe’s emitted code

Timer resolution bounds what can be claimed about the shortest CloudSC loops. Ice supersaturation adjustment runs in roughly 16 to 19  $\mu$ s, so the differences between its variants are only a few timer ticks wide and should not be read as ordering the variants. There is a need for an update in DaCe’s timer such that finer granularity on smaller loop tests can be undertaken.

More kernels can be looked at, especially kernels in the TSVC-2 package that only C++ could vectorize and DaCe failed on either compiler, these could be compared against situations where C++ fails vs DaCe’s success to see where the benefit of DaCe’s emitted code is and improvement.

Separately, DaCe should investigate why GCC’s behavior changes so sharply in response to `-fassociative-math` on DaCe-emitted code when the resulting assembly shows no floating-point changes that would be associated with such a flag. The flag appears to be buying GCC’s willingness to attempt the transformation rather than permitting a riskier one, and if that is right, the same effect may be reachable without relaxing FP semantics at all.

## 6 Appendix

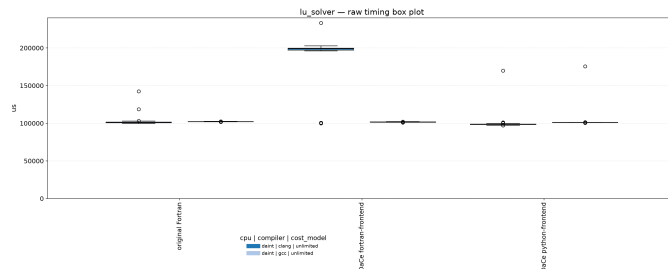


Figure 19: Boxplot of `Lu_solver` with unlimited cost-model, showing the drastic increase in execution time for an unlimited clang vectorized run

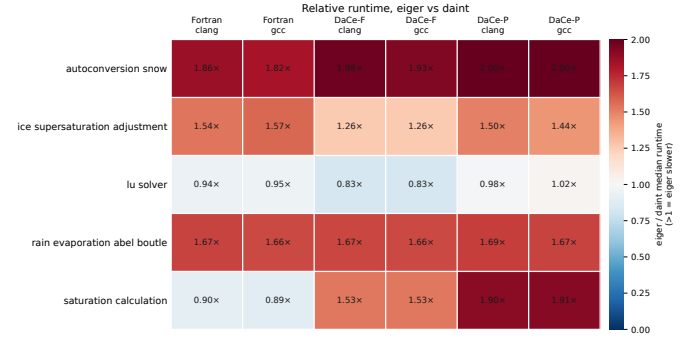


Figure 20: Heatmap showing how `Eiger/x86/amd_epyc` is significantly worse than `Daint/aarch64/arm_grace`. The only exception being `Lu_solver`. The older CPU is not as prevalent anymore and thus the focus of the study should stay with `arm_grace`

## References

- [1] D. Callahan, J. Dongarra, and D. Levine, “Vectorizing compilers: A test suite and results,” in *Proceedings of the 1988 ACM/IEEE Conference on Supercomputing*, ser. Supercomputing ’88. IEEE Computer Society Press, 1988, pp. 98–105.
- [2] S. Maleki, Y. Gao, M. J. Garzarán, T. Wong, and D. A. Padua, “An evaluation of vectorizing compilers,” in *Proceedings of the 2011 International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’11. IEEE, 2011, pp. 372–382.
- [3] UoB-HPC, “TSVC\_2: An updated C version of the test suite for vectorising compilers,” [https://github.com/UoB-HPC/TSVC\\_2](https://github.com/UoB-HPC/TSVC_2), accessed: 2026-08-23.
- [4] A. Pohl, B. Cosenza, and B. H. H. Juurlink, “Portable cost modeling for auto-vectorizers,” in *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2019, pp. 359–369.
- [5] S. Siso, W. Armour, and J. Thiyagalingam, “Evaluating auto-vectorizing compilers through objective withdrawal of useful information,” *ACM Transactions on Architecture and Code Optimization*, vol. 16, no. 4, pp. 40:1–40:23, 2019.
- [6] A. Pohl, B. Cosenza, and B. Juurlink, “Vectorization cost modeling for NEON, AVX and SVE,” *Performance Evaluation*, vol. 140–141, p. 102106, 2020.
- [7] T. Chen, J. Wu, F. Yin, and X. Ren, “Automatic generation of cross-platform vectorization kernels for cloud microphysics parameterization,” *Journal of Parallel and Distributed Computing*, vol. 213, p. 105273, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731526000511>
- [8] CSCS, “Alps hardware,” <https://docs.cscs.ch/alps/hardware/>, accessed: 2026-08-23.
- [9] —, “Daint,” <https://docs.cscs.ch/clusters/daint/>, accessed: 2026-08-23.
- [10] NVIDIA, “NVIDIA Grace Hopper Superchip architecture in-depth,” <https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/>, accessed: 2026-08-23.
- [11] CSCS, “Eiger,” <https://docs.cscs.ch/clusters/eiger/>, accessed: 2026-08-23.
- [12] Loop invariant code motion. [Online]. Available: <https://llvm.org/docs/Passes.html>
- [13] “Clang 24.0.0git documentation.” [Online]. Available: <https://clang.llvm.org/docs/ClangCommandLineReference.html>